

- Description

This module manages a smart machine vibration monitoring system, ensuring that:

1. The system continuously monitors the vibration level of a machine (in mm/s).
2. If the vibration level exceeds a predefined threshold, an alert is triggered to indicate potential equipment issues and the need for maintenance.
3. If the vibration level is within the safe range, the machine is considered to be operating normally.

This system helps prevent equipment failure and ensures timely maintenance by continuously monitoring machine vibration levels.

- Specification

module SmartVibrationMonitoring

var currentVibration, maxVibrationThreshold: real

process updateVibration(newVibration: real) returns (currentVibration: real)

functional scenario form:

currentVibration > maxVibrationThreshold && output "ALERT: Vibration level (" + currentVibration + " mm/s) exceeds threshold (" + maxVibrationThreshold + " mm/s). Maintenance required.", Vibration exceeds threshold; alert triggered.

||

currentVibration <= maxVibrationThreshold && output "Vibration level (" + currentVibration + " mm/s) is within safe limits.", Vibration within safe range; no action required.

end process

end module

- Java Code

```
package code;
public class c101_SmartVibrationMonitor {
    private double maxVibrationThreshold;
    private double currentVibration;
    public c101_SmartVibrationMonitor(double maxVibrationThreshold, double initialVibration) {
        this.maxVibrationThreshold = maxVibrationThreshold;
        this.currentVibration = initialVibration;
    }
    public void updateVibration(double newVibration) {
        currentVibration = newVibration;
        System.out.printf("Current Vibration: %.2f mm/s\n", currentVibration);
        if (currentVibration > maxVibrationThreshold) {
            System.out.println("ALERT: Vibration level too high! Maintenance required.");
        } else {
```

```

        System.out.println("Vibration level is within safe limits."
);
    }
}
public double getCurrentVibration() {
    return currentVibration;
}
public static void main(String[] args) {
    c101_SmartVibrationMonitor monitor = new c101_SmartVibrationMonitor(20.0, 15.0);
    // testcase-VT:
    //     monitor.updateVibration(18.0);
    //     monitor.updateVibration(22.5);
    //     monitor.updateVibration(19.0);

    // testcase-FT:
    //     monitor.updateVibration(9.063142030601998E290);
    //     monitor.updateVibration(2.3292275018647134E293);
    //     monitor.updateVibration(6.7769769832148005E302);

    // testcase-Z3:
    //     monitor.updateVibration(18.0);
    //     monitor.updateVibration(22.5);
    //     monitor.updateVibration(19.0);

    // testcase-UVT:
    monitor.updateVibration(18.0);
    monitor.updateVibration(22.5);
    monitor.updateVibration(19.0);
}
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c101_SmartVibrationMonitor.java	100.0 %	2	0	2
▼ c101_SmartVibrationMonitor	100.0 %	2	0	2
● updateVibration(double)	100.0 %	2	0	2
● c101_SmartVibrationMonitor(do		0	0	0
§ main(String[])		0	0	0

3. Vibration testing instruction coverage: 94.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.2 %	54	21,788	21,842
▼ src	0.2 %	54	21,788	21,842
▼ code	0.2 %	54	21,788	21,842
▼ c101_SmartVibrationMonitor.java	94.7 %	54	3	57
▼ c101_SmartVibrationMonitor	94.7 %	54	3	57
● updateVibration(double)	100.0 %	29	0	29
● c101_SmartVibrationMonitor(do	100.0 %	9	0	9
● main(String[])	100.0 %	16	0	16

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.1 %	1	841	842
▼ src	0.1 %	1	841	842
▼ code	0.1 %	1	841	842
▼ c101_SmartVibrationMonitor.java	50.0 %	1	1	2
▼ c101_SmartVibrationMonitor	50.0 %	1	1	2
● updateVibration(double)	50.0 %	1	1	2
● c101_SmartVibrationMonitor(do		0	0	0
● main(String[])		0	0	0

5. Fuzz testing instruction coverage: 89.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.2 %	51	21,791	21,842
▼ src	0.2 %	51	21,791	21,842
▼ code	0.2 %	51	21,791	21,842
▼ c101_SmartVibrationMonitor.java	89.5 %	51	6	57
▼ c101_SmartVibrationMonitor	89.5 %	51	6	57
● updateVibration(double)	89.7 %	26	3	29
● c101_SmartVibrationMonitor(do	100.0 %	9	0	9
● main(String[])	100.0 %	16	0	16

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c101_SmartVibrationMonitor.java	100.0 %	2	0	2
▼ c101_SmartVibrationMonitor	100.0 %	2	0	2
● updateVibration(double)	100.0 %	2	0	2
● c101_SmartVibrationMonitor(do		0	0	0
● main(String[])		0	0	0

7. Z3 instruction coverage: 94.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.2 %	54	21,788	21,842
▼ src	0.2 %	54	21,788	21,842
▼ code	0.2 %	54	21,788	21,842
▼ c101_SmartVibrationMonitor.java	94.7 %	54	3	57
▼ c101_SmartVibrationMonitor	94.7 %	54	3	57
● updateVibration(double)	100.0 %	29	0	29
● c101_SmartVibrationMonitor(do	100.0 %	9	0	9
● main(String[])	100.0 %	16	0	16

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c101_SmartVibrationMonitor.java	100.0 %	2	0	2
▼ c101_SmartVibrationMonitor	100.0 %	2	0	2
● updateVibration(double)	100.0 %	2	0	2
● c101_SmartVibrationMonitor(do		0	0	0
● main(String[])		0	0	0

9. Unimproved V-method instruction coverage: 94.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.2 %	54	21,788	21,842
▼ src	0.2 %	54	21,788	21,842
▼ code	0.2 %	54	21,788	21,842
▼ c101_SmartVibrationMonitor.java	94.7 %	54	3	57
▼ c101_SmartVibrationMonitor	94.7 %	54	3	57
● updateVibration(double)	100.0 %	29	0	29
● c101_SmartVibrationMonitor(do	100.0 %	9	0	9
● main(String[])	100.0 %	16	0	16